



Haramaya University
Building the Basis for Development

**COLLEGE OF COMPUTING AND INFORMATICS
DEPARTMENT OF SOFTWARE ENGINEERING
SOFTWARE TESTING AND QUALITY ASSURANCE
GROUP ASSIGNMENT
YEAR-IV | SEM-II | SECTION-A**

Members

- 1. Meti Yonas.....0935/15**
- 2. Tewodros Fikadu...1000/15**
- 3. Tolasa Tesfaye.....1010/15**
- 4. Tesfaye Eshetu.....0996/15**
- 5. Worku Tashome.....1036/15**
- 6. Dagimawit Kebede...0277/15**

JANUARY 2, 2026
HARAMAYA UNIVERSITY

Question #1: Defect Life Cycle in Testing

A. Discuss the defect lifecycle in a software project.

The defect life cycle in software testing: is the path traced by the bug from the moment of identification to its resolution.

The purpose of Defect life cycle is to easily coordinate and communicate current status of defect which changes to various assignees and make the defect fixing process systematic and efficient.

Stages of defect Life Cycle in Testing

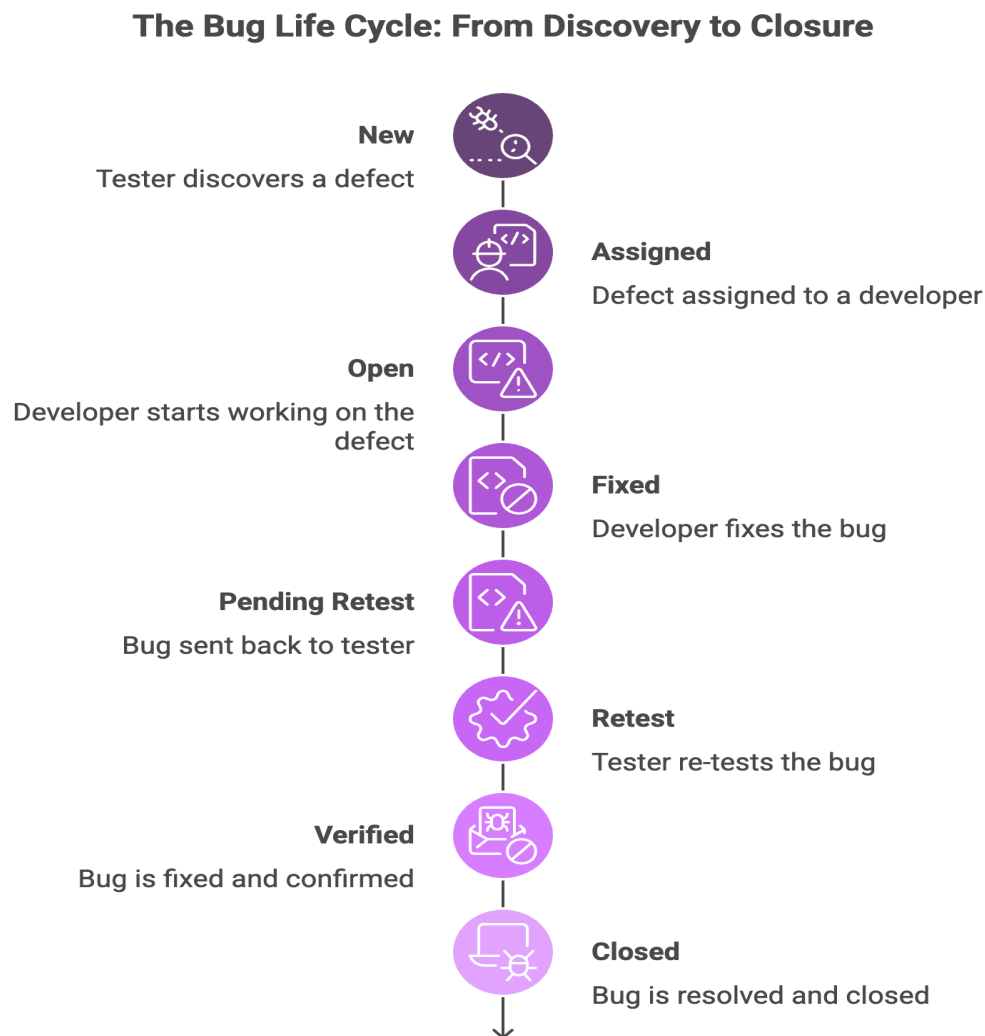


Figure 1: Defect life cycle

1. New

When a tester discovers a defect during the testing phase, they log it into a bug tracking system like Jira. At this point, the defect is in the "New" state, awaiting review. Essential information such as environment details, the test case that failed, and steps to reproduce the defect are included in the bug report.

2. Assigned

Once the defect is posted, it gets assigned to the development team. The project manager or lead developer reviews the bug and assigns it to a team member for resolution. The status is changed from "New" to "Assigned."

3. Open

The developer starts working on the defect, changing the status to "Open." At this stage, the developer will investigate the bug and determine the necessary code changes to fix it.

4. Fixed

When the developer fixes the bug, the status is updated to "Fixed." The necessary changes are made, and the bug is ready for further testing.

5. Pending Retest

After the bug is marked "Fixed," it is sent back to the tester for testing. The tester checks whether the defect was truly fixed by executing the relevant test cases again. The bug is now in the "Pending Retest" stage.

6. Retest

In this stage, the tester re-tests the bug using manual or test automation techniques to verify if the bug is resolved. If the test case passes, the bug can move to the next stage.

7. Verified

If the bug is fixed and the test case no longer fails, the tester changes the status to "Verified." This confirms that the issue is no longer present in the software.

8. Closed

Once the bug is "Verified," it is marked as "Closed," signifying that the bug is resolved. If the bug fixes the issue without any problems, no further action is needed.

Other Possible States of a Bug

While the main stages define the general flow, there are other statuses that a bug can assume during its life cycle:

- ❖ **Reopened:** If the bug persists even after the developer fixes it, the tester can mark it as "Reopened," sending it back to the developer for further action.
- ❖ **Deferred:** In some cases, a bug may not be critical for the current release and is deferred to be fixed in the next release.
- ❖ **Rejected or Not a Bug:** If the development team determines that the reported issue is not a bug (e.g., it may be due to a misunderstanding of requirements), the status is marked as "Rejected" or "Not a Bug."

B. If a large number of defects repeatedly move from Resolved to Reopened, what does this imply about the defect resolution and verification process?

If a large number of defects repeatedly move from **Resolved** to **Reopened**, it indicates the following issues in the defect resolution and verification process:

- ❖ The defect fix is **not complete** or does not fully solve the problem.
- ❖ The **root cause** of the defect was not properly identified.
- ❖ Developers may have **misunderstood the defect requirements** or expected behavior.
- ❖ Poor or unclear **communication** between testers and developers.
- ❖ Defects are marked as resolved **without proper testing or validation**.
- ❖ Verification and retesting activities may be **rushed or insufficient**.

This leads to:

- Rework increases.
- Project timelines are delayed.
- Overall software quality and team confidence are reduced.

C. If most defects are closed without entering the Pending Retest state, what risks might this pose to software quality?

If most defects are **closed without entering the Pending Retest** state, the following risks may affect software quality:

- ❖ **Defects may not be properly verified** before closure.
- ❖ **Unfixed or partially fixed defects** could remain in the system.

- ❖ **Regression issues** might go unnoticed in related areas of the software.
- ❖ **False confidence in software stability**, leading to premature release.
- ❖ **Increased defect leakage** into production, affecting end-users.
- ❖ **Higher maintenance costs** later due to undiscovered defects.

This leads to:

- Reduced overall software quality.
- Poor user satisfaction and trust.

Question #2: Software Test and Quality Metrics.

a. Failure Rate:

Test failure rate is a metric used in software engineering to measure the percentage of failed test cases during a specific build or over a particular time period. It is calculated by dividing the number of failed tests by the total number of tests executed, then multiplying the result by 100 to get a percentage.

This metric provides insight into the reliability and stability of the software under development.

b. Mean Time Between Failures (MTBF)

Mean time between failure (MTBF) is a measure of the reliability of a system or component. It's a crucial element of maintenance management, representing the average time that a system or component will operate before it fails.

c. Mean Time to Repair (MTTR)

MTTR is the average time it would take to restore the uptime of a given component or system. MTTR is used to optimize repair times.

d. Availability Percentage

Availability Percentage is a key metric that measures the proportion of time a system, service, or application is operational, functional, and accessible to users.

It is essentially the measure of a system's *uptime* compared to its total planned operational time.

Availability can be calculated either using Total Uptime or Total Downtime.

a. Using Total Uptime

$$\text{Availability (\%)} = \frac{\text{Total Uptime}}{\text{Total Elapsed Time}} \times 100$$

Total Uptime = the time the system is available and working correctly

Total Elapsed Time = Total time observed (Uptime + Downtime)

b. Using Total Downtime

$$\text{Availability (\%)} = \frac{\text{Total Elapsed Time} - \text{Total Downtime}}{\text{Total Elapsed Time}} \times 100$$

Total Elapsed Time – Total Downtime = Total Uptime

e. Test Coverage

Test coverage shows how much of your application has been tested. It helps teams see whether their testing efforts have reached all the important parts of the software. That includes code, user flows, edge cases, and business requirements.

Types of Test Coverage

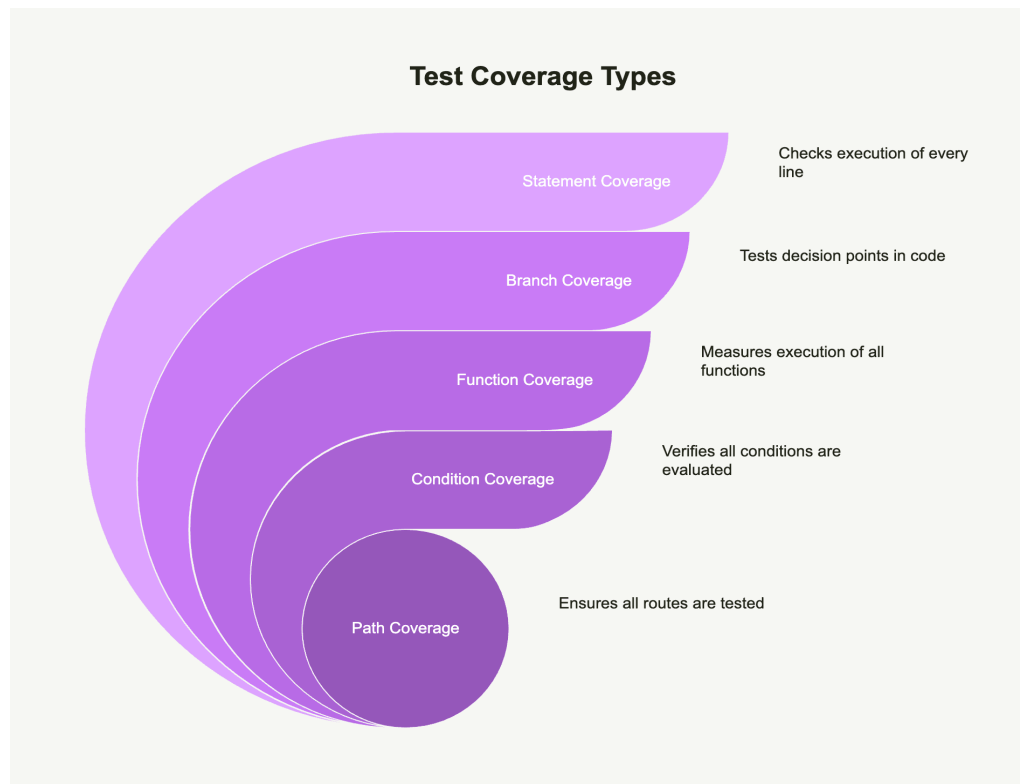


Figure 2: Test Coverage Type

f. Test pass or Fail Rate

The test pass rate is a measure used in software engineering to determine the proportion of test cases that have passed during a particular build or over a specific period.

$$\text{Test Pass Rate (\%)} = \frac{\text{Passed Tests}}{\text{Executed Tests}} \times 100$$

Importance of Test Pass Rate

- ❖ **Quality Assurance:** High pass rate shows software meets functional and performance standards, ensuring reliability and user satisfaction.
- ❖ **Project Health:** Tracks development progress; declining rates signal issues needing attention.
- ❖ **Resource Allocation:** Helps plan testing and development effort efficiently.

The test failure rate is intrinsically linked to the test pass rate as it measures the percentage of test cases that fail.

$$\text{Test Fail Rate (\%)} = \frac{\text{Number of Test Cases Failed}}{\text{Total Number of Test Cases Executed}} \times 100$$

g. Defect Density

Defect density is a mathematical value that indicates the number of flaws found in software or other parts over the period of a development cycle.

It helps to evaluate software quality and identify high risk areas

$$\text{Defect Density} = \frac{\text{Number of Defects Found}}{\text{Size of the Software}}$$

h. Defect Severity Index (DSI)

Defect Severity Index measures the **overall impact of defects** in a software system by considering their severity levels. It helps in identifying how serious the defects are and prioritizing fixes to improve software quality.

$$\text{DSI} = \frac{\sum(\text{Severity of Each Defect} \times \text{Number of Defects at That Severity})}{\text{Total Number of Defects}}$$

i. Defect Age

Defect Age is the **total time a defect remains open** in the system, measured from the moment it is reported until it is resolved or closed. It helps track how long defects stay in the system and can indicate efficiency of the development and testing process.

$$\text{Defect Age} = \text{Defect Closed Date} - \text{Defect Reported Date}$$

j. Defect Resolution Time (DRT)

Defect Resolution Time measures the **average time taken to fix a defect** from the moment it is reported until it is resolved. It helps assess the efficiency of the development and QA team in addressing defects.

$$\text{Defect Resolution Time} = \frac{\sum(\text{Time to Resolve Each Defect})}{\text{Total Number of Defects Resolved}}$$

k. Defect Removal Efficiency (DRE)

Defect Removal Efficiency (DRE) provides a measure of how effectively defects are removed during development and testing before release.

This metric shows the percentage of defects caught before the software goes live compared to the total defects found (before and after release).

l. Defect leakage: refers to bugs or issues that are not detected during the testing phase but are discovered only after the software is released into production or in later testing phases.

It highlights gaps in the QA/testing process when defects slip through earlier testing stages. [[BrowserStack](#)]

m. Customer Satisfaction Index (CSI) is a metric representing overall customer feedback about their experience with a product or service, often calculated from surveys that measure satisfaction levels across different aspects of the product.

This concept reflects how satisfied customers are with the software or service offered. [[Qualtrics](#)]

$$\text{CSI} = \frac{\text{Sum of Customer Satisfaction Scores}}{\text{Total Number of Respondents}}$$

n. Net Promoter Score

Net Promoter Score (NPS) is an indicator used to measure customer loyalty and the likelihood that customers will recommend a brand, product, or service to others.

It is based on responses to the question: “How likely are you to recommend this product to a friend?”[[Qualtrics](#)]

Question #3: Car insurance premium calculation.

Test Scenario:

Verify that the car insurance premium discount is applied correctly based on the customer's age, driving experience and accident history.

Conditions

C1: Age > 25

C2: Driving Experience > 5 years

C3: No accidents in the past year

Each condition will be tested independently for True and False, following Condition Coverage testing.

Test Case ID	Pre condition	Test Steps	Test Data	Expected Result	Actual Result	Status
TC1	Customer account exists	1. Enter Age = 28 2. Enter Driving Experience = 7 3. Select No Accidents = True 4. Submit form	Age = 28, Exp = 7, No Accidents = True	Discount applied	Discount Applied	Pass
TC2	Customer account exists	1. Enter Age = 22 2. Enter Driving Experience = 7 3. Select No Accidents = True 4. Submit form	Age = 22, Exp = 7, No Accidents = True	Discount NOT applied(Age condition false)	Discount NOT applied	Pass
TC2	Customer account exists	1. Enter Age = 28 2. Enter Driving Experience = 4 3. Select No Accidents = True 4. Submit form	Age = 28, Exp = 4, No Accidents = True	Discount NOT applied(Experience condition false)	Discount NOT applied	Pass
TC4	Customer account exists	1. Enter Age = 28 2. Enter Driving Experience = 7 3. Select No Accidents = False 4. Submit form	Age = 28, Exp = 7, No Accidents = False	Discount Not Applies(Accident condition false)	Discount NOT applied	Pass

Question #4: Equivalence Partitioning and Boundary Value Analysis – Public Transport Fare System

Scenario:

Verify that the fare calculator applies the correct fare based on a passenger's age, including handling invalid inputs.

Conditions:

Age Group	Fare
1-12(Child)	\$15
13-18(Youth)	\$30
19-59(Adult)	\$75
60+(senior)	\$40

Equivalence Partition

Divide the input (age) into **valid partitions** where all values in a partition are treated the same. Also consider **invalid partitions** (ages outside 1–120, e.g., negative numbers or unrealistically high numbers).

Partition Type	Age Range	Expected Fare
Valid-Child	1-12	\$15
Valid-Youth	13-18	\$30
Valid -Adult	19-59	\$75
Valid-Senior	60+	\$40
Invalid- Negative	<1	Error
Invalid Too High	>120	Error

Boundary Value Analysis

- Test **values at the edges** of partitions, since errors often occur at boundaries.
- Include **just below, at, and just above** boundary values.

Age Group	Lower Boundary	Upper Boundary	BVA Test Values
Child	1	12	0,1,2,11,12,13
Youth	13	18	12,13,14,17,18,19
Adult	19	59	18,19,20,58,59,60
Senior	60	-	59,60,61,120,121

Test Case

Test Case ID	Pre condition	Test Steps	Test Data	Expected Result	Actual Result	Status
TC01	Fare system is running	1. Open fare system 2. Enter age 9 3. Submit	Age = 9	Fare Displayed as \$15(Child)	Fare Displayed as \$15	Pass
TC02	Fare system is running	1. Enter age 14 2. Submit	Age = 14	Fare Displayed as \$30(Youth)	Fare Displayed as \$30	Pass
TC03	Fare system is running	1. Enter age 17 2. Submit	Age = 17	Fare Displayed as \$30(Youth)	Fare Displayed as \$30	Pass
TC04	Fare system is running	1. Enter age 21 2. Submit	Age = 21	Fare Displayed as \$75(Adult)	Fare Displayed as \$75	Pass
TC05	Fare system is running	1. Enter age 58 2. Submit	Age = 58	Fare Displayed as \$75(Adult)	Fare Displayed as \$75	Pass
TC06	Fare system is running	1. Enter age 64 2. Submit	Age = 64	Fare Displayed as \$40(Senior)	Fare Displayed as \$40	Pass
TC07	Fare system is running	1. Enter age -2 2. Submit	Age = -2	System should reject input	Error message displayed	Pass
TC08	Fare system is running	1. Enter age 135 2. Submit	Age = 135	System should reject input	Error message displayed	Pass

Question #5: The Haramaya University Integrated Student Information Management System (HU-ISIM)

The Haramaya University Integrated Student Information Management System (HU-ISIMS) is used by students for critical academic services such as course registration, grade viewing, and personal information management. Based on continuous usage of the system over years, a number of quality-related issues have been observed. As a Software Quality Assurance student, we identified some issues and proposes possible improvements to enhance the overall quality and usability of the system

1. Limited Accessibility (Campus-Only Access)

Observed Problem: HU-ISIMS can only be accessed using the university's internal Wi-Fi network. Students are unable to access the system from outside the campus or using mobile data, which limits flexibility and convenience.

Proposed Solution:

- ✓ Enable secure off-campus access using VPN or role-based authentication.
- ✓ Implement secure web access with proper encryption and access control.
- ✓ Allow limited external access for non-critical services such as grade viewing

2. System Congestion During High User Load

Observed Problem: When a large number of users access the system simultaneously (especially during registration or result announcement), the system becomes busy, slow, or unresponsive.

Quality Attribute Affected: Performance, Scalability, Reliability

Proposed Solution:

- ✓ Improve server capacity and implement load balancing.
- ✓ Optimize database queries and backend processing.
- ✓ Perform load and stress testing before peak academic periods.

3. Lack of Data Editability for Students

Observed Problem: Some personal or academic information entered by students cannot be edited online. To correct such data, students must physically visit the system administrator, which is time-consuming.

Quality Attribute Affected: Maintainability, Usability, Flexibility

Proposed Solution:

- ✓ Allow controlled self-service editing with approval workflows.
- ✓ Enable students to submit edit requests electronically.
- ✓ Maintain audit logs for any data changes to ensure data integrity.

4. **Dependency on Manual Administrative Intervention**

Observed Problem: Many issues such as incorrect data, registration problems, or access errors require direct intervention from system administrators.

Quality Attribute Affected: Efficiency, Maintainability

Proposed Solution:

- ✓ Automate common administrative tasks.
- ✓ Introduce a ticketing or request-tracking system.
- ✓ Provide clear system guidance and FAQs to reduce admin workload.

5. **Poor User Experience Under Constraints**

Observed Problem: Due to limited access, system congestion, and rigid data control, the overall user experience is frustrating for students, especially during critical academic timelines.

Quality Attribute Affected: User Satisfaction, Usability

Proposed Solution:

- ✓ Improve system responsiveness and accessibility.
- ✓ Gather student feedback regularly and incorporate improvements.
- ✓ Conduct usability and user experience (UX) testing.

The quality audit of HU-ISIMS highlights key issues related to accessibility, performance, flexibility, and usability. Restricting access to campus Wi-Fi, system overload during peak usage, and limited data editability reduce the effectiveness of the system. By improving accessibility, enhancing system performance, and introducing controlled self-service features, HU-ISIMS can significantly improve its quality and provide a more efficient and user-friendly experience for students.

References

https://bugbug.io/blog/software-testing/bug-life-cycle/?utm_source=chatgpt.com#stages-of-the-defect-life-cycle

<https://www.software.com/engineering-metrics/test-failure-rate>

<https://www.ibm.com/think/topics/mtbf>

<https://www.geeksforgeeks.org/software-testing/software-testing-defect-density/>

https://www.browserstack.com/guide/defect-leakage?utm_source=chatgpt.com